

Programación II

DESARROLLO DE UNA AGENDA MÉDICA

Estudiantes:
Brittany Elizondo

Emanuel Pérez

Saúl Borbón.

Docente: Jason Muñoz Rojas.

Fecha de Entrega: 13 de Octubre del 2026.

Introducción

El presente proyecto detalla el diseño y la planificación de una aplicación de escritorio orientada a la gestión automatizada de una agenda médica con control de turnos y disponibilidad. En el ámbito de la salud, la optimización del tiempo de los profesionales y la asignación eficiente de espacios para los pacientes son factores críticos para la calidad del servicio. Esra solución tecnologica busca sustituir los métodos tradicionales de agendamiento mediante un sistema robusto desarrollado en el lenguaje C# con persistencia de datos en el motor MySQL.

Definición del Problema

Actualmente los centros médicos que carecen de un sistema automatizado de agenda enfrentan serios inconvenientes operativos. El agendamiento manual o mediante herramientas genéricas provoca problemas recurrentes como el solapamiento de citas para un mismo médico, la asignación de pacientes a horarios donde el profesional no está disponible y la dificultad de gestionar multiples especialidades de forma organizada. Asimismo la falta de una arquitectura de software definida provoca que los datos sean vulnerables y difíciles de mantener, lo que impacta en la toma de decisiones y el seguimiento clínico del paciente.

Justificación

El desarrollo de esta aplicación se justifica en la necesidad de proveer una herramienta centralizada y segura que agilice los procesos de recepción y administración médica. Al implementar reglas de negocio automatizadas en el código, se mitigan por completo los errores humanos en la asignación de turnos. Desde el punto de vista académico, el proyecto permite aplicar conceptos avanzados de la Programación Orientada a Objetos, el diseño de bases de daros relacionales y la separación de responsabilidades mediante una arquitectura de tres capas (UI, BL, DAL).

Objetivos

Objetivo General: Desarrollar una aplicación de escritorio en C# para la gestión de una agenda médica, utilizando programación orientada a objetos, arquitectura por capas y persistencia de información en MySQL, aplicando reglas de negocio para garantizar la disponibilidad y consistencia de la información.

Objetivos Específicos:

- Diseñar e implementar una base de datos relacional en MySQL que garantice la integridad de los datos de médicos, pacientes y citas.
- Implementar una arquitectura organizada coo mínimo en tres capas: Presentación (UI), lógica de negocio (BL), y acceso a datos (DAL).
- Aplicar validaciones y reglas de negocio para evitar el solapamiento de horarios y garantizar la correcta gestión de los estados de las citas.

Alcance del proyecto

El sistema cubrirá los siguientes módulos y capacidades para la gestión clínica:

- **Módulo de médicos:**
- **Módulo de pacientes**
- **Módulo de Especialidades:** Gestión de las ramas de la medicina disponibles en el centro y vinculación de múltiples especialidades a un mismo médico.
- **Módulo de Citas y Agenda:** Creación, reprogramación, cancelación y consulta de citas filtradas por médico, fecha, paciente y estado de la consulta.
- **Módulo de Reportes:** Generación de métricas básicas sobre citas atendidas, canceladas y acumulado por día.

Requisitos funcionales y reglas del negocio

Esta sección describe las capacidades operativas mínimas que poseerá el sistema y las restricciones lógicas que la capa de negocio garantizará para el correcto funcionamiento de la agenda médica.

Requisitos funcionales mínimos (RF)

- **RF-01. Gestión de Médicos:** El sistema permitirá registrar, modificar, consultar y deshabilitar la información de los profesionales de la salud. Esto incluye datos obligatorios como identificación (cédula), nombre completo, información de contacto, especialidades asignadas, estado operativo (Activo/Inactivo) y sus respectivos horarios de atención semanal.
- **RF-02. Gestión de Pacientes:** El sistema proveerá interfaces para el control total (CRUD) de los datos de los pacientes, permitiendo almacenar su identificación, nombre, apellidos, teléfono, correo electrónico y fecha de nacimiento.
- **RF-03. Gestión de Especialidades:** El módulo permitirá dar de alta y administrar las diferentes ramas médicas de la clínica. Asimismo, habilitará la funcionalidad para asociar una o múltiples especialidades a un mismo médico.

- **RF-04. Gestión de Citas**: El núcleo del sistema permitirá la creación de nuevas citas, reprogramación de fechas u horarios, cancelaciones definitivas y la consulta del estado en tiempo real de cada bloque de tiempo guardado.
- **RF-05. Consulta de Agenda Médica:** La aplicación dispondrá de un panel de búsqueda avanzada para filtrar y consultar la agenda médica utilizando simultáneamente o por separado los criterios de: médico, fecha específica, paciente y estado de la cita.
- **RF-06. Usuarios y Roles**: El sistema implementará un control de acceso seguro mediante credenciales de inicio de sesión. Se contemplarán de forma obligatoria los roles de Administrador (acceso total a configuración y reportes) y Recepción (permisos limitados a la operación de citas y pacientes).
- **RF-07. Reportes Básicos**: El sistema generará reportes y consultas rápidas en pantalla que agruparán: citas agendadas por día y por médico, el histórico de citas canceladas y el listado de pacientes efectivamente atendidos.

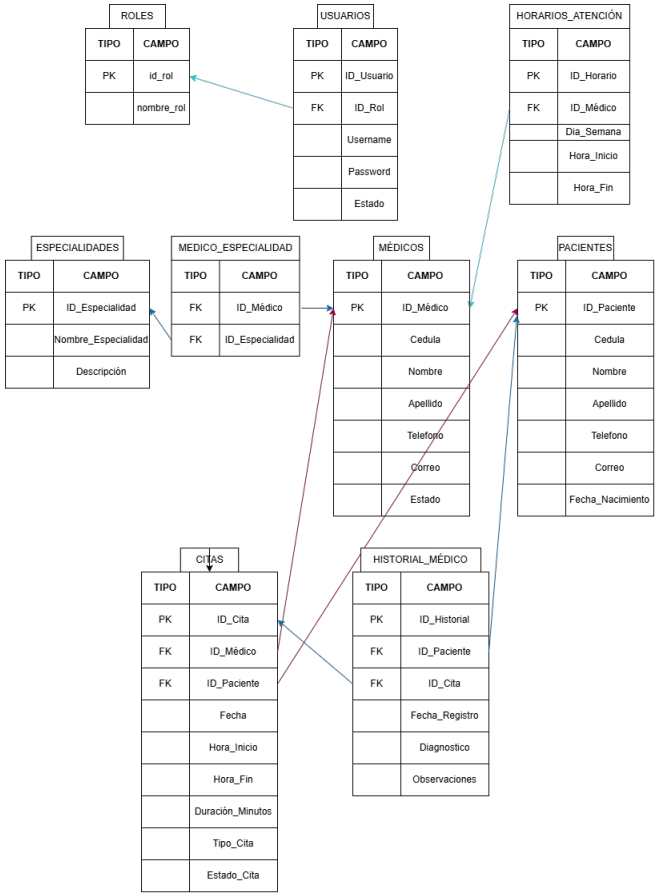
Reglas de negocio (RN)

- **RN-01. Horario del Médico:** El sistema validará estrictamente que la hora de inicio y finalización de una cita médica se encuentre dentro del rango horario de atención definido y aprobado para el médico seleccionado.
- **RN-02. Prevención de Solapamientos:** No se permitirá, bajo ninguna circunstancia, registrar dos citas en un mismo bloque de tiempo para un mismo médico. La base de datos y la lógica bloquearán cualquier choque de horarios.
- **RN-03. Conflicto de Pacientes:** El sistema bloqueará el agendamiento si un paciente intenta reservar una cita en un horario donde ya posee otra cita programada con un médico diferente en el mismo centro médico.
- **RN-04. Duración Parametrizada:** La duración de cada espacio en la agenda no será fija ni manual. El sistema obligará a seleccionar bloques predefinidos de 30, 45 o 60 minutos según la naturaleza de la consulta.
- **RN-05. Tipo de Cita:** Al registrar un turno, se deberá especificar obligatoriamente una de las tres modalidades soportadas por la clínica: Presencial, Virtual o Urgencia.

- **RN-06. Integridad de Información:** Para evitar la pérdida de relaciones necesarias o inconsistencias en la base de datos, no se permitirá la eliminación física de registros que contengan dependencias (por ejemplo, borrar un médico que posee citas amarradas o un paciente con historial médico registrado).
- **RN-07. Ciclo de Estados de la Cita**: Toda cita guardada en el sistema deberá controlar y actualizar su situación operativa transitando únicamente por los siguientes estados válidos: Programada, Confirmada, Atendida, Cancelada, No asistió y Reprogramada.
- **RN-08. Principio de Herencia:** En el diseño orientado a objetos del código (capa BL), la herencia se aplicará de forma estricta únicamente cuando exista una relación de generalización real que la justifique (por ejemplo, modelar una clase base Persona de la cual hereden Medico y Paciente), evitando su uso con el único fin de cumplir un requisito sintáctico.

Diseño de la base de datos (Modelo entidad-relación)

Descripción del modelo: La base de datos relacional está diseñada en MySQL y consta de 9 tablas interconectadas para evitar la redundancia de información y asegurar la integridad referencial.



Arquitectura de la solución

El proyecto se desarrollará utilizando un patrón de diseño arquitectónico en tres capas (UI, BL, DAL). Esta estructura garantiza la separación de responsabilidades, lo que significa que cada capa se encarga exclusivamente de una tarea específica, facilitando el mantenimiento del código, el aislamiento de errores y la escalabilidad del sistema.

A continuación, se detallan los tres componentes que conforman la arquitectura y sus respectivas responsabilidades dentro del sistema de agenta médica.

Capa de presentación o Interfaz de Usuario (UI)

- • **Responsabilidad:** Es la capa externa del software y el único punto de contacto con el usuario final. Estará construida utilizando Windows Forms y su función principal es capturar las entradas del usuario (clics, textos, fechas) a través de los formularios y mostrar los resultados de forma visual.
- **Regla de diseño**: Esta capa tiene estrictamente prohibido comunicarse de manera directa con la base de datos MySQL o ejecutar validaciones de lógica médica. Su único deber es tomar los datos de las pantallas, enviárselos a la capa de lógica (BL) y esperar una respuesta para desplegar mensajes o alertas al usuario.

Capa de Lógica de Negocio (BL)

- **Responsabilidad:** Funciona como el cerebro de la aplicación y el intermediario entre la pantalla y los datos. Su función es procesar toda la información recibida de la UI y aplicar estrictamente las Reglas de Negocio (RN-01 a RN-08).
- **Regla de diseño**: Aquí es donde se programará el código que calcula si una cita se solapa con otra, si el médico está en su horario laboral o si un paciente ya tiene un turno reservado a la misma hora. Si las reglas se cumplen, le pide a la capa de datos (DAL) que guarde la información; si no se cumplen, frena el proceso y lanza una excepción o alerta hacia la UI.

Capa de acceso a datos (DAL)

- **Responsabilidad:** Es la capa más interna del sistema y se encarga exclusivamente de la persistencia de la información. Su único objetivo es gestionar la conexión física con el motor de base de datos MySQL.
- **Regla de diseño:** Contiene el código necesario para ejecutar las operaciones CRUD (Crear, Leer, Actualizar y Borrar) mediante consultas SQL parametrizadas. Esta capa no sabe nada de

reglas médicas ni de botones en la pantalla; ella solo recibe objetos de datos desde la BL, los traduce a comandos SQL, los inyecta en la base de datos y devuelve los resultados crudos.

Arquitectura propuesta

- Enfoque general.

  El sistema se va a desarrollar como una aplicación de escritorio en C#, usando Windows Forms y una base de datos MySQL. En teoría a idea es que el programa permita manejar médicos, pacientes, especialidades y citas médicas de manera ordenada.

  Se trabajará con una arquitectura de tres capas: Una capa para la interfaz, otra para la lógica del sistema y la última para el acceso a los datos.

- Diseño de la Interfaz (UI).

  Contendrá formularios para cada módulo:

  - Médicos: Registrar, editar y consultar información.
  - Pacientes: Almacenar datos personales
  - Especialidades: Listar y asociar con los médicos
  - Citas: Crear, reprogramar, cancelar y consultar
  - Usuarios/Roles: Acceso según permisos

- Lógica de negocio (BL).

  Esta parte es la que se encarga de aplicar las reglas del sistema:

  - Verificar disponibilidad de los médicos.
  - Evitar solapamiento de citas.
  - Controlar horarios
  - Manejar los estados de las citas (Programada, reprogramada, cancelada)
  - Validar que los datos estén completos

- Acceso a datos (DAL).

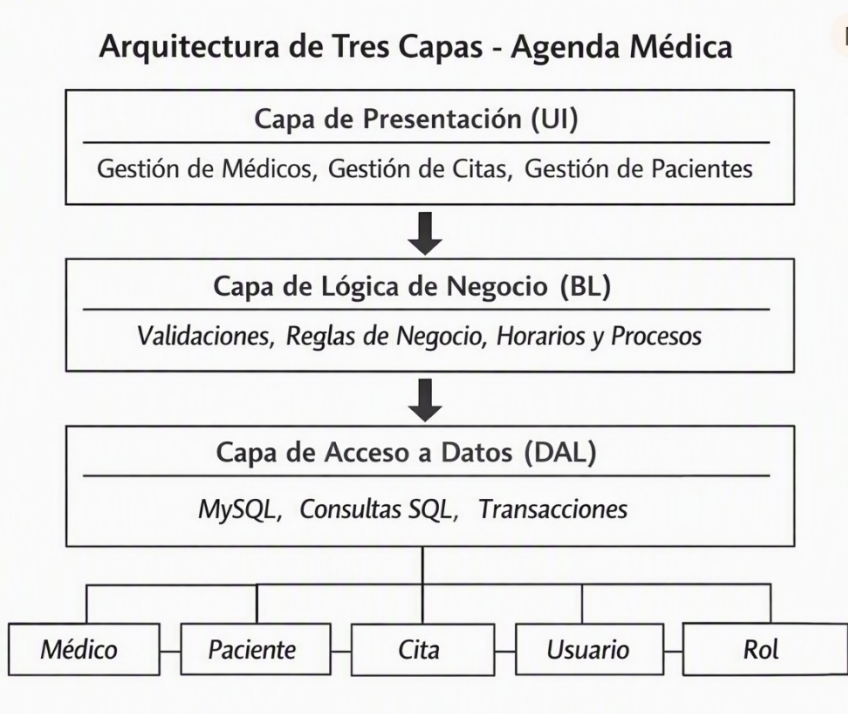
  La base de datos será MySQL y tendrá tablas relacionadas entre sí.

  Algunas principales:

  - Roles
  - Usuarios
  - Médicos
  - Pacientes
  - Citas
  - Especialidad
  - Historial médico

- Flujo general del sistema.

  1. El usuario inicia sesión según su rol
  2. Accede al módulo que requiere
  3. Realiza operaciones CRUD con validaciones automáticas.
  4. El sistema verifica la disponibilidad y evita errores con el horario
  5. Se actualizan los estados y registro en la base de datos



Estructura Inicial del Proyecto en C#

A continuación se detalla el árbol de directorios y la organización de archivos creada en la solución de Visual Studio para dar cumplimiento a la arquitectura de tres capas exigida.

- **Solución: AgendaMedica**
  - AgendaMedica.UI
    - Program.cs
    - FromLogin.cs
    - FromPrincipal.cs
  - AgendaMedica.BL
    - Modelos/
      - Modelo.cs, Paciente.cs, Cita.cs, Usuario.cs
    - Controladores/
  - AgendaMedica.DAL
    - ConexiónBD.cs
    - Repositorios/

Repositorio de control de versiones

Para el desarrollo progresivo del software y el cumplimiento de las buenas prácticas de ingeniería se ha configurado un repositorio en GitHub.
https://github.com/saulborbon/AgendaMedica_Proyecto